CS231N Midterm Cheatsheet

1 Image Classification & Linear Classifiers

Notation: $x_i \in \mathbb{R}^D$ = i-th image flattened to a vector (D=HWC); $y_i \in \{1,..,C\}$ = its label; C = #classes; N = #train examples; $W \in \mathbb{R}^{C\times D}$ = weight matrix (one row $W_k \in \mathbb{R}^D$ per class); $b \in \mathbb{R}^C$ = bias; $s = Wx + b \in \mathbb{R}^C$ = vector of class scores (s_k = score of class k); $p \in \Delta^C$ = softmax probabilities (sum to 1); λ = regularisation strength; L = loss. *Symbols*: $a^\top$ = transpose (row ↔ column); $a^\top b = \sum_i a_i b_i$ = dot product; $a\,b^\top$ = outer product (matrix); $\mathbb{1}[\cdot]$ = indicator (1 if true, else 0); e_k = one-hot vector with a 1 in position k; $\nabla_w L$ = gradient of L w.r.t. w; $\frac{\partial L}{\partial W}$ = matrix of partial derivatives, same shape as W. **Data split**: train / val (hyperparams) / test (touched once). k-fold CV: split train into k folds, average val acc.

1.1 k-Nearest Neighbors

Predict label of x by majority vote over k nearest training points.

$$d_{L1}(x,x') = \sum_p |x_p - x'_p|, \quad d_{L2}(x,x') = \sqrt{\sum_p (x_p - x'_p)^2}$$

L_2 rotation-invariant; L_1 axis-dependent. Train $O(1)$, test $O(N)$. Bad for raw pixels (curse of dim.).

1.2 Linear Classifier

Scores $s = Wx + b$, $W \in \mathbb{R}^{C\times D}$, $x \in \mathbb{R}^D$. Predicted class $\hat{y} = \arg\max_k s_k$. Bias trick: $\tilde{x} = [x;1] \in \mathbb{R}^{D+1}$, $\tilde{W} = [W\ b] \in \mathbb{R}^{C\times(D+1)} \Rightarrow s = \tilde{W}\tilde{x}$. Views: **algebraic** (matmul); **visual** (each row W_k = template, $s_k = W_k^\top x$); **geometric** (hyperplane $W_k^\top x + b_k = 0$). Decision boundary between i,j: $(W_i - W_j)^\top x + (b_i - b_j) = 0$.

1.3 Loss functions

Multiclass SVM (hinge), margin Δ: penalise every wrong class j whose score is within Δ of the correct class y_i.

$$L_i = \sum_{j\neq y_i} \max(0,\ s_j - s_{y_i} + \Delta), \qquad L = \frac{1}{N}\sum_i L_i + \lambda R(W)$$

Subgradient w.r.t. rows of W (each $W_k \in \mathbb{R}^D$, so $\frac{\partial L_i}{\partial W_k} \in \mathbb{R}^D$ has the shape of one row):

- Wrong-class row $j \neq y_i$: $\frac{\partial L_i}{\partial W_j} = \mathbb{1}[\,s_j - s_{y_i} + \Delta > 0\,]\cdot x_i^\top$
  (margin violated ⇒ push W_j away from x_i).
- Correct-class row: $\frac{\partial L_i}{\partial W_{y_i}} = -\big(\sum_{j\neq y_i} \mathbb{1}[\,s_j - s_{y_i} + \Delta > 0\,]\big)\, x_i^\top$
  (pull W_{y_i} toward x_i, scaled by # violators).

Here $x_i^\top \in \mathbb{R}^{1\times D}$ is the input written as a row so the result has the row-shape of W_k. **Softmax / Cross-Entropy**: convert scores to probs, penalise − log of the true-class prob.

$$p_k = \frac{e^{s_k}}{\sum_j e^{s_j}}, \qquad L_i = -\log p_{y_i} = -s_{y_i} + \log\sum_j e^{s_j}$$

Gradient on scores is the "probs minus one-hot" rule:

$$\frac{\partial L_i}{\partial s_k} = p_k - \mathbb{1}[k{=}y_i] \quad \Rightarrow \quad \frac{\partial L_i}{\partial W} = (p - e_{y_i})\, x_i^\top$$

($p - e_{y_i} \in \mathbb{R}^C$ is a column, $x_i^\top \in \mathbb{R}^{1\times D}$ a row, so the outer product has shape $C\times D$ matching W.) Numerical stability: replace s_k by $s_k - \max_j s_j$ before exponentiating. At init ($W \approx 0$): $L_i \approx \log C$ (CE), $L_i \approx C-1$ (SVM, Δ=1). Softmax never = 0; SVM hits 0 once margin satisfied. **Regularizers**: $R_{L2}(W) = \sum W_{ij}^2$ ($\nabla = 2W$, weight decay), $R_{L1} = \sum |W_{ij}|$ (sparse), elastic net. SVM+L_2 ⇒ max-margin: minimize $\frac{1}{2}\|W\|^2$ s.t. correct margin.

2 Optimization

Notation: w = parameter vector (weights), η = learning rate, $L(w)$ = loss, $g_t = \nabla_w L_{B_t}(w_{t-1})$ = stochastic gradient on minibatch B_t at step t, v = velocity (momentum buffer, same shape as w), G or v_t = running 2nd-moment of g (per-param, elementwise), m_t = running 1st-moment, ρ, β_1, β_2 = decay rates, ε = small const (10^{-8}) for numerical safety, $\odot$ = elementwise, $H = \nabla^2 L$ = Hessian. Gradient: $\nabla_w L$, steepest-descent direction $-\nabla L/\|\nabla L\|$. **Numerical (centered)**: $\frac{\partial L}{\partial w_i} \approx \frac{L(w+he_i) - L(w-he_i)}{2h}$, $h \sim 10^{-5}$, error $O(h^2)$. Relative err vs analytic: $\frac{|g_a - g_n|}{\max(|g_a|,|g_n|)} < 10^{-7}$ ok.

SGD (mini-batch): $w \leftarrow w - \eta\, g$, $|B| \in [32, 512]$. **Momentum** (v = velocity, init 0): $v \leftarrow \rho v - \eta g$; $w \leftarrow w + v$; $\rho \approx 0.9$ friction. Equiv form: $v = \rho v + g$; $w = w - \eta v$. **Nesterov**: $v \leftarrow \rho v - \eta \nabla L(w + \rho v)$; $w \mathrel{+}= v$ (grad evaluated at lookahead point). **AdaGrad** (G = sum of squared grads, per-param): $G \mathrel{+}= g \odot g$; $w \mathrel{-}= \eta\, g/(\sqrt{G} + \varepsilon)$. LR shrinks monotonically. **RMSProp**: replace G by EMA of g^2: $G \leftarrow \beta G + (1-\beta) g \odot g$; $w \mathrel{-}= \eta g/(\sqrt{G} + \varepsilon)$, $\beta \approx 0.99$. **Adam** = momentum + RMSProp + bias-correction. m_t = EMA of g (1st mom), v_t = EMA of g^2 (2nd mom), both init 0:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t \odot g_t$$
$$\hat{m}_t = m_t/(1-\beta_1^t), \quad \hat{v}_t = v_t/(1-\beta_2^t)$$
$$w_t = w_{t-1} - \eta\, \hat{m}_t/(\sqrt{\hat{v}_t} + \varepsilon)$$

Defaults: β_1=0.9, β_2=0.999, ε=10^{-8}, η=10^{-3}. Bias-correction critical at start (else m, v biased toward 0). AdamW: decouple weight decay, add $w \mathrel{-}= \eta\lambda w$ outside the $\sqrt{\hat{v}}$ rescaling.

LR schedules (t = step, T = total, T_w = warmup): step $\eta_t = \eta_0 \gamma^{\lfloor t/s \rfloor}$; exp $\eta_t = \eta_0 e^{-kt}$; $1/t$: $\eta_t = \eta_0/(1+kt)$; cosine $\eta_t = \frac{1}{2}\eta_0\left(1 + \cos\frac{t\pi}{T}\right)$; warmup linear $\eta_t = \eta_0\, t/T_w$ for $t < T_w$. **Linear scaling rule**: $\eta \propto |B|$ (Goyal et al.). **Newton**: $w \leftarrow w - H^{-1}\nabla L$ (H = Hessian); infeasible at scale. L-BFGS approximates H^{-1} from past grads, full-batch only.

3 Neural Networks

Notation: x = layer input, W_ℓ, b_ℓ = weights/bias of layer ℓ, a_ℓ=$W_\ell h_{\ell-1} + b_\ell$ = pre-activation, h_ℓ=$\sigma(a_\ell)$ = activation, σ = nonlinearity, n_{in}, n_{out} = fan-in / fan-out of a layer. 2-layer MLP: $h = \sigma(W_1 x + b_1)$, $s = W_2 h + b_2$. Universal approx (1 hidden, enough units).

Activations & derivatives:

- Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$; $\sigma' = \sigma(1-\sigma) \le 1/4$.
- $\tanh(z) = 2\sigma(2z) - 1$; $\tanh' = 1 - \tanh^2 z$.
- ReLU $\mathrm{ReLU}(z) = \max(0, z)$; $\mathrm{ReLU}' = \mathbb{1}[z > 0]$.
- Leaky $\max(\alpha z, z)$, α=0.01; ELU $z (z \ge 0)$, $\alpha(e^z - 1)$ else.
- GELU $\approx z\,\Phi(z)$; Swish $z\sigma(z)$.
- Softplus $\log(1 + e^z)$, $' = \sigma(z)$.

Preprocessing: $\tilde{x} = (x - \mu)/\sigma$ per-channel; PCA whitening $\tilde{x} = \Lambda^{-1/2} U^\top (x - \mu)$. **Init**: Xavier $\mathrm{Var}(W) = 1/n_{in}$ (or $2/(n_{in}+n_{out})$, tanh); He $\mathrm{Var}(W) = 2/n_{in}$ (ReLU). Sample $W_{ij} \sim \mathcal{N}(0, \mathrm{Var})$.

3.1 Backpropagation

Chain rule on a DAG. For $z = f(x,y)$: $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$. **Gate gradients**:

- add: $z = x + y \Rightarrow \frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}$ (distributor).
- mul: $z = xy \Rightarrow \frac{\partial L}{\partial x} = y\frac{\partial L}{\partial z}$, $\frac{\partial L}{\partial y} = x\frac{\partial L}{\partial z}$ (swap).
- max: routes upstream grad to argmax input.
- branch (copy): sums incoming grads.
- exp: $z = e^x \Rightarrow \frac{\partial L}{\partial x} = z\frac{\partial L}{\partial z}$.
- div: $z = x/y \Rightarrow \frac{\partial L}{\partial x} = \frac{1}{y}\frac{\partial L}{\partial z}$, $\frac{\partial L}{\partial y} = -\frac{x}{y^2}\frac{\partial L}{\partial z}$.

Matrix forms: $y = Wx + b$:

$$\frac{\partial L}{\partial x} = W^\top \frac{\partial L}{\partial y}, \quad \frac{\partial L}{\partial W} = \frac{\partial L}{\partial y}\, x^\top, \quad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial y}$$

Batched ($Y = XW^\top + \mathbf{1}b^\top$, $X \in \mathbb{R}^{N\times D}$): $\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y}W$, $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Y}^\top X$, $\frac{\partial L}{\partial b} = \sum_n \frac{\partial L}{\partial Y_n}$. **Softmax+CE** fused: $\frac{\partial L}{\partial s} = p - y$ (one-hot). **Sigmoid+BCE**: $\frac{\partial L}{\partial s} = \sigma(s) - y$.

4 Convolutional Networks

Notation: C_{in}, C_{out}=K = in/out channels, H, W = spatial in, H', W' = spatial out, F = filter (kernel) size, S = stride, P = zero-pad each side, N = batch size. Input $N \times C_{in} \times H \times W$; K filters each $C_{in} \times F \times F$; stride S, pad P.

Output shape $N \times K \times H' \times W'$ with

$$H' = \left\lfloor \frac{H - F + 2P}{S} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$$

Same pad: $P = (F-1)/2$ keeps $H', W' = H, W$ when S=1. **Valid**: P=0. **# params (weights+bias)**:

$$\#\text{params} = K\,(C_{in}F^2 + 1)$$

 # mult-add (per input):

$$\#\text{MAC} = H'\,W' \cdot K \cdot C_{in}F^2$$

(#FLOPs ≈ 2 · #MAC; multiply by N for a batch.) **Activation size**: $N\,K\,H'W'$ values.

Pool $F \times F$ stride S (no pad): $H' = (H-F)/S + 1$; **0 params**, #MAC=0 (just compares/avgs). Max-pool grad routes to argmax. **FC layer** $D_{in} \to D_{out}$: params $= D_{in}D_{out} + D_{out}$, MACs $= D_{in}D_{out}$. 1×1 **conv** ($C_{in} \to C_{out}$, spatial $H \times W$): params $C_{in}C_{out} + C_{out}$, MACs $HWC_{in}C_{out}$ — channel mix / bottleneck. **Depth-wise sep** ($F \times F$, $C \to C'$): MACs $H'W'(CF^2 + CC')$ vs std $H'W'CC'F^2$, factor $\approx 1/C' + 1/F^2$. **Stack of two** 3×3 (channels C): RF 5, params $2 \cdot 9C^2 = 18C^2$ vs one 5×5 $25C^2$. **Receptive field**: stack L convs $F \times F$ stride 1 ⇒ RF=$1 + L(F-1)$. With strides: $\mathrm{RF}_\ell = \mathrm{RF}_{\ell-1} + (F_\ell - 1)\prod_{i<\ell} S_i$. **Conv as matmul (im2col)**: unroll patches into columns, single GEMM. **Conv backward**: input grad = full-conv of dout with flipped filter; weight grad = correlation of input with dout.

5 CNN Architectures

LeNet-5: $[C{-}P] \times 2 \to$ FC. **AlexNet** (2012): 5 conv + 3 FC, 60M params, ReLU + dropout(0.5) + L2 + augmentation. **VGG-16**: only 3×3, S=1, P=1 conv & 2×2, S=2 pool; 138M params (mostly FC). **GoogLeNet/Inception**: parallel 1×1, 3×3, 5×5,pool concatenated; 1×1 bottleneck; GAP head; aux classifiers; 5M params. **ResNet**: residual $y = F(x) + x$. Bottleneck 1×1→3×3→1×1. Identity shortcut ⇒ gradient $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y}(I + F'(x))$ has identity term (no vanish). **ResNeXt**: grouped conv. **DenseNet**: $x_\ell = H([x_0, \ldots, x_{\ell-1}])$. **SENet**: channel attention $s = \sigma(W_2\,\mathrm{ReLU}(W_1\,\mathrm{GAP}(x)))$; $x' = s \odot x$. **MobileNet**: depthwise sep = depthwise conv ($C_{in}F^2$ per out pix) + pointwise 1×1 ($C_{in}C_{out}$): factor $\approx 1/C_{out} + 1/F^2$ FLOPs.

5.1 Normalization

Let $x \in \mathbb{R}^{N\times C\times H\times W}$. **BN**: per-channel over (N,H,W): $\hat{x} = (x - \mu)/\sqrt{\sigma^2 + \varepsilon}$, $y = \gamma\hat{x} + \beta$ ($\gamma, \beta \in \mathbb{R}^C$). Train uses batch stats; test uses running mean/var $\mu_r \leftarrow (1-\alpha)\mu_r + \alpha\mu_B$. BN backward: $\frac{\partial L}{\partial x_i} = \frac{1}{m\sigma}[m\,d\hat{x}_i - \sum_j d\hat{x}_j - \hat{x}_i\sum_j d\hat{x}_j\hat{x}_j]$ where $d\hat{x} = \gamma\,dy$. **LN**: per-sample over (C,H,W) (Transformers). **IN**: per-sample-per-channel over (H,W) (style transfer). **GN**: split C into G groups; over $(C/G, H, W)$ — batch independent.

5.2 Regularization & generalization

Dropout (train): $\tilde{h} = h \odot m$, $m_i \sim \mathrm{Bern}(p)$; rescale by $1/p$ (inverted dropout) so test is plain. Equiv to ensembling 2^n subnets. **Data aug**: flips, crops, color jitter, mixup $\tilde{x} = \lambda x_i + (1-\lambda)x_j$, label likewise; cutout. **Label smoothing**: $y_k = (1-\epsilon)\mathbb{1}[k{=}y] + \epsilon/C$. **Early stop** when val loss rises.

5.3 Transfer learning

Feature extract: freeze backbone, train new head (η normal). Fine-tune: unfreeze top, $\eta \sim 10^{-4}$ to avoid wrecking pretrained weights. Layer-wise LR decay common.

6 RNNs / LSTM / GRU

Notation: x_t = input at step t, h_t = hidden state (memory), c_t = LSTM cell state, y_t = output, W_{hh}, W_{xh}, W_{hy} = recurrent / input / output weight matrices, $\odot$ = elementwise. Gates i, f, o, g are vectors of same dim as h.

Idea: process a sequence one step at a time, carrying a hidden vector h_t that summarises everything seen so far. *Same* weights W are reused at every step (parameter sharing across time ⇒ handles variable-length input, generalises across positions). Recurrence:

$$h_t = f_W(h_{t-1}, x_t), \quad y_t = g_W(h_t)$$

Unrolling in time gives a deep feed-forward net of depth T with tied weights; train with *backprop through time* (BPTT). Input/output patterns: one-to-many (captioning), many-to-one (sentiment), many-to-many aligned (tagging) or seq2seq (translation).

Vanilla RNN: $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$, $y_t = W_{hy}h_t$. At each step we mix the previous summary h_{t-1} with the new input x_t through one linear layer + nonlinearity.

BPTT: $L = \sum_t L_t$, $\frac{\partial L}{\partial W_{hh}} = \sum_t \sum_{k\le t} \frac{\partial L_t}{\partial h_t}\left(\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}\right)\frac{\partial h_k}{\partial W_{hh}}$, with $\frac{\partial h_j}{\partial h_{j-1}} = \mathrm{diag}(1 - \tanh^2)\, W_{hh}^\top$. Gradient is a *product* of T Jacobians ⇒ if largest singular value of $W_{hh} < 1$ it shrinks (vanish, can't learn long deps), if > 1 it blows up (explode, NaNs). **Truncated BPTT**: backprop only K steps to bound cost. **Fixes**: clip gradient if $\|g\| > \theta$: $g \leftarrow g\,\theta/\|g\|$ (explode); use gated units below (vanish).

6.1 LSTM

Idea: add a separate *cell state* c_t that acts as a memory "conveyor belt". At each step, learnable gates decide what to *forget* from old memory (f), what *new* content to write (i, g), and how much memory to *output* as the hidden state (o). All gates are sigmoids in $(0,1)$ acting as soft on/off switches; g is the candidate update (tanh).

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} + b \right)$$

$$c_t = f \odot c_{t-1} + i \odot g, \qquad h_t = o \odot \tanh(c_t)$$

Why it works: the cell update is *additive*, so $\frac{\partial c_t}{\partial c_{t-1}} = \mathrm{diag}(f)$ — backprop along the cell path is just elementwise multiplication by forget gates, no repeated matmul ⇒ gradient "highway" that preserves long-range signal. Forget-gate bias init to 1 so early in training memory is kept by default.

GRU (lighter LSTM, no separate cell): reset gate $r_t = \sigma(\cdot)$ controls how much of the past to use when proposing $\tilde{h}$; update gate $z_t = \sigma(\cdot)$ interpolates between old state and new candidate. $\tilde{h} = \tanh(W_h x_t + U_h(r \odot h_{t-1}))$, $h_t = (1-z) \odot h_{t-1} + z \odot \tilde{h}$.

6.2 LM, captioning, seq2seq

Char/word LM: at each step predict next token $p(x_t | x_{<t}) = \mathrm{softmax}(W_{hy}h_t)$, train by teacher forcing (feed true x_t, not the model's prediction). Sample with temperature τ: $p \propto \exp(s/\tau)$ ($\tau < 1$ peaky/greedy-ish, $\tau > 1$ more uniform). **Beam search** keeps top-B partial sequences by log-prob. **Captioning**: CNN encodes image to feature v, used as h_0 or concatenated to each decoder input; RNN generates caption word-by-word with teacher forcing in train, sampling at test. **Seq2Seq**: encoder RNN compresses source into final state h_T^E that initialises decoder h_0^D — single-vector bottleneck loses info on long inputs; **attention** fixes this by letting the decoder look back at all encoder states each step.

7 Attention & Transformers

Notation: $X \in \mathbb{R}^{N\times d}$ = sequence of N tokens, dim d. Q,K,V = queries/keys/values (rows are tokens). d_k = per-head key dim. h = #heads. W^Q, W^K, W^V, W^O = learned projection matrices. PE = positional encoding.

Idea: replace recurrence by direct, content-based lookup between every pair of tokens. Each token emits a *query* q_i asking "what info do I need?", a *key* k_j advertising "what info I have", and a *value* v_j carrying that info. Attention weight $A_{ij} \propto \exp(q_i^\top k_j/\sqrt{d_k})$ is a soft, differentiable dictionary lookup; the output is $\sum_j A_{ij} v_j$. Unlike RNNs every token sees every other in $O(1)$ steps ⇒ much better long-range modeling and full parallelism over t.

Scaled dot-product: $Q \in \mathbb{R}^{N_q\times d_k}, K \in \mathbb{R}^{N_k\times d_k}, V \in \mathbb{R}^{N_k\times d_v}$

$$A = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big) \in \mathbb{R}^{N_q\times N_k}, \quad \text{Attn} = AV \in \mathbb{R}^{N_q\times d_v}$$

Softmax is row-wise (each query's weights sum to 1). Why $\sqrt{d_k}$: if q,k iid $\mathcal{N}(0,1)$ then $\text{Var}(q\cdot k)=d_k$; without rescaling logits grow with d_k, softmax saturates, gradient on losing keys vanishes.

Self-attention: $Q=XW^Q$, $K=XW^K$, $V=XW^V$ from the *same* sequence; permutation-equivariant (no notion of order ⇒ need PE). Complexity $O(N^2 d)$ time, $O(N^2)$ memory in the attention matrix — the main scaling bottleneck. **Cross-attention**: Q from one sequence (e.g. decoder), K,V from another (encoder) ⇒ used to read from a memory. **Masked / causal SA**: set $A_{ij}=-\infty$ for $j>i$ pre-softmax so token i only attends to $\le i$; needed for autoregressive decoding so train (parallel, all positions) matches test (one token at a time). **Padding mask**: same trick to ignore padding tokens in variable-length batches.

Multi-head (h heads, $d_k=d_v=d/h$): $\text{MHA}(X) = [\text{head}_1,\dots,\text{head}_h]W^O$, $\text{head}_i = \text{Attn}(XW_i^Q, XW_i^K, XW_i^V)$. Same total compute as one big head, but each head can specialise (syntactic, positional, semantic relations) — like multiple filters in a conv layer. $W^O \in \mathbb{R}^{d\times d}$ mixes heads.

Why PE: SA is permutation-equivariant, so $f(\pi X)=\pi f(X)$ — without position info "dog bites man" = "man bites dog". Add $PE \in \mathbb{R}^{N\times d}$ to token embeddings. **Sinusoidal** (fixed, extrapolates to longer seqs):

$$PE_{p,2i} = \sin(p/10000^{2i/d}),\ PE_{p,2i+1} = \cos(p/10000^{2i/d})$$

Different frequencies ⇒ relative positions can be expressed as linear functions of PE. **Learned** PE: a trainable table (length-limited). **RoPE**: rotates each (q,k) pair of dims by angle $p\theta_i$ inside the dot product ⇒ encodes *relative* position directly in $q^\top k$. **Transformer block** (Pre-LN, modern default):

$$x' = x + \text{MHA}(\text{LN}(x))$$
$$y = x' + \text{MLP}(\text{LN}(x'))$$

Residuals ⇒ gradient highway + each sublayer learns a refinement. **LayerNorm** (over feature dim, per token) stabilises across variable-length / batch-independent settings. Pre-LN trains stably without warmup; original Post-LN (x+Sublayer(x) then LN) needs warmup. MLP (a.k.a. FFN): $\text{MLP}(z) = W_2\,\text{GELU}(W_1 z + b_1) + b_2$, hidden $d_{\text{ff}}\sim 4d$. Acts position-wise, mixing channels (attention mixes tokens, MLP mixes features). **Decoder block**: masked SA → cross-attn (Q from decoder, K,V from encoder) → MLP, all with residual+LN.

Param count per block $\approx 4d^2$ (QKVO) + $2\cdot d\cdot d_{\text{ff}}=8d^2$ (MLP) ⇒ $12d^2$. Plus 2 LN ($2d$ each) and biases. **FLOPs** per block (seq N): MHA $\approx 4Nd^2$ (proj) + $2N^2d$ (attn matmuls); MLP $\approx 8Nd^2$. **KV cache** (inference): cache K,V from past tokens ⇒ each new token costs $O(Nd)$ instead of $O(N^2 d)$; memory grows linearly in N. **Efficient variants**: sparse / local-window / Longformer / BigBird ($O(N)$ or $O(N\log N)$); Linformer / Performer (low-rank or kernel approx); FlashAttention (exact, IO-aware tiling, no N^2 materialisation).

Training recipes: AdamW (β_1=0.9, β_2=0.95–0.999), weight decay 0.05–0.1 on non-LN/non-bias params, linear warmup then cosine decay, label smoothing 0.1, dropout 0.1, gradient clip 1.0, large batch via accumulation. Init: small std (e.g. 0.02); often scale residual branch by $1/\sqrt{2L}$.

Common families:

- **Encoder-only** (BERT, ViT): bidirectional SA; classification / representation. BERT pretrain = MLM (predict 15% masked tokens) + NSP.
- **Decoder-only** (GPT, LLaMA): causal SA; trained on next-token prediction; generation by sampling.
- **Encoder-decoder** (T5, original Transformer): translation, seq2seq; cross-attn lets decoder read encoder memory.

ViT: image $H\times W\times 3$ split into $N=HW/P^2$ non-overlapping patches (e.g. P=16); flatten each to $3P^2$, linear embed to d; prepend learned [CLS] token; add (learned) PE; pass through standard Transformer encoder; classify from [CLS] output. Conv inductive bias (locality, translation equivariance) is gone ⇒ needs large pretraining (JFT) or strong aug+distillation (DeiT). **Swin**: hierarchical, attention inside shifted local windows ⇒ linear in image size, brings back locality. **DETR** (see §Detection): set-prediction Transformer over CNN features, removes NMS/anchors.

8 Object Detection

Notation: A,B = bounding boxes, GT = ground truth, fg/bg = foreground/background, anchor = predefined reference box, (x_a,y_a,w_a,h_a) = anchor center+size, t_* = regression targets, p_t = predicted prob of true class. **IoU**$(A,B) = |A\cap B|/|A\cup B|$. TP if IoU$\ge$ 0.5 (PASCAL) or averaged over [0.5 : 0.05 : 0.95] (COCO). **Precision**$=\frac{TP}{TP+FP}$, **Recall**$=\frac{TP}{TP+FN}$. AP $=\int_0^1 P(R)\,dR$; mAP avg over classes. **NMS**: while boxes left: pick highest score box b^*, remove all b with IoU$(b,b^*)>\tau$. **Bbox regression target** (Fast R-CNN): given anchor (x_a,y_a,w_a,h_a) and GT (x,y,w,h):

$$t_x=\tfrac{x-x_a}{w_a},\ t_y=\tfrac{y-y_a}{h_a},\ t_w=\log\tfrac{w}{w_a},\ t_h=\log\tfrac{h}{h_a}$$

Loss: smooth L_1: $\text{sL}_1(z) = \begin{cases}0.5z^2 & |z|<1\\ |z|-0.5 & \text{else}\end{cases}$ **R-CNN**: SS→2k crops→CNN per crop→SVM+regress. Slow. **Fast R-CNN**: CNN once → RoI Pool → FC heads (cls + reg). Multi-task $L = L_{cls} + \lambda[\text{fg}]L_{loc}$. **Faster R-CNN**: + RPN with k anchors per location; 4-task loss. End-to-end. **Mask R-CNN**: + per-RoI FCN mask head; RoI Align (bilinear, no quantization). **One-stage**: dense predict at every grid cell over anchors. **Focal loss**: $FL(p_t) = -\alpha_t(1-p_t)^\gamma \log p_t$, $\gamma\sim 2$ down-weights easy bg. **DETR**: Transformer enc-dec + N object queries; bipartite Hungarian matching loss; no anchors/NMS.

9 Segmentation & Visualization

Semantic per-pixel softmax CE. **FCN**: Conv → 1×1 to C classes → upsample. **U-Net**: encoder-decoder + skip concat. **Upsampling**: nearest, bilinear, transposed conv (output size $= S(H-1)+F-2P$), unpooling (saved indices). Transposed conv = forward of conv backward; can cause checkerboard (use F divisible by S).

Saliency: $M_p = \max_c |\frac{\partial s_y}{\partial x_{p,c}}|$. **Guided BP**: in ReLU backward keep grad only if both input>0 *and* dout>0. **Grad-CAM**: $\alpha_k = \frac{1}{Z}\sum_{ij}\frac{\partial s_y}{\partial A^k_{ij}}$; map $L = \text{ReLU}(\sum_k \alpha_k A^k)$. **Activation max**: $x^* = \arg\max_x f_i(x) - \lambda\|x\|^2$ via grad ascent. **DeepDream**: ascent on $\|f_\ell(x)\|^2$. **Style transfer**: features $F^\ell \in \mathbb{R}^{C\times HW}$; Gram $G^\ell = F^\ell (F^\ell)^\top \in \mathbb{R}^{C\times C}$.

$$L = \alpha\sum_\ell \|F^\ell - F_c^\ell\|^2 + \beta\sum_\ell w_\ell \|G^\ell - G_s^\ell\|^2$$

FGSM: $x' = x + \varepsilon\,\text{sign}(\nabla_x L(x,y))$. PGD: iterate + project to ℓ_∞ ball.

10 Video Understanding

Input $T\times 3\times H\times W$. **Single-frame CNN**: classify each frame, average. **Late fusion**: per-frame CNN → pool over T → FC. **Early fusion**: stack T frames in channel dim ($3T$ channels) at first conv. **3D CNN**: kernel $T'\times F\times F$; output $(T-T'+2P)/S+1$ in time. Params $K\,C_{in}\,T'F^2$. **C3D**: 3D VGG-style. **I3D**: inflate 2D ImageNet weights along time ($W^{3D} = W^{2D}/T'$). **Two-stream**: spatial RGB CNN + temporal optical-flow CNN, fuse. **CNN+LSTM**: per-frame features into LSTM. **SlowFast**: two pathways — Slow (low fps, large C), Fast (high fps, small C); lateral connections. **ViViT/TimeSformer**: factorized space then time attention.

11 Worked Backprop Examples

11.1 Sigmoid CE — single neuron

$z = w^\top x + b$, $p=\sigma(z)$, $L = -[y\log p + (1-y)\log(1-p)]$. $\frac{\partial L}{\partial p} = \frac{p-y}{p(1-p)}$, $\frac{\partial p}{\partial z} = p(1-p) \Rightarrow \frac{\partial L}{\partial z} = p-y$. $\frac{\partial L}{\partial w} = (p-y)x$, $\frac{\partial L}{\partial b} = p-y$.

11.2 2-layer net (softmax CE)

Forward: $h = \text{ReLU}(W_1 x + b_1)$, $s = W_2 h + b_2$, $p = \text{softmax}(s)$, $L = -\log p_y$. Backward:

$$ds = p - e_y$$
$$dW_2 = ds\,h^\top,\ db_2 = ds$$
$$dh = W_2^\top ds$$
$$da_1 = dh \odot \mathbb{1}[a_1>0] \quad (a_1 = W_1x+b_1)$$
$$dW_1 = da_1\,x^\top,\ db_1 = da_1$$

With L2 reg $\lambda\|W\|^2$ add $2\lambda W$ to each dW.

11.3 Convolution backward (sketch)

$Y = X * W$ ($*$ = correlation). With upstream dY: $dW = X * dY$ (correlation), $dX = dY \,\tilde{*}\, \text{flip}(W)$ (full conv). With im2col: forward $Y_{col} = W_{row}\,X_{col}$; $dW_{row} = dY_{col}\,X_{col}^\top$, $dX_{col} = W_{row}^\top dY_{col}$, then col2im.

11.4 Max-pool backward

For each window route dY to the position of the max in the forward; others 0.

11.5 LSTM backward — key piece

With $a = W[h_{t-1};x_t] + b$ split into a_i, a_f, a_o, a_g: $do = dh_t \odot \tanh c_t$, $dc_t{+}= dh_t \odot o \odot (1-\tanh^2 c_t)$. $df = dc_t \odot c_{t-1}$; $di = dc_t \odot g$; $dg = dc_t \odot i$; $dc_{t-1} = dc_t \odot f$. Then through gate nonlinearities: $da_{i,f,o} = d(i,f,o)\odot\sigma(1-\sigma)$; $da_g = dg\odot(1-g^2)$.

11.6 Attention backward (sketch)

$S = QK^\top/\sqrt{d_k}$, $A = \text{softmax}(S)$, $O = AV$. $dV = A^\top dO$, $dA = dO\,V^\top$. $dS_{ij} = A_{ij}(dA_{ij} - \sum_k A_{ik}dA_{ik})$ (softmax-Jacobian row). $dQ = dS\,K/\sqrt{d_k}$, $dK = dS^\top Q/\sqrt{d_k}$.

12 Useful Derivations & Numbers

Softmax Jacobian: $p=\text{softmax}(s) \Rightarrow \frac{\partial p_i}{\partial s_j} = p_i(\delta_{ij} - p_j)$; with CE: $\frac{\partial L}{\partial s} = p - y$.

CE = NLL: $L = -\sum_k y_k \log p_k$; one-hot $\Rightarrow L = -\log p_{y^*}$; $= \text{KL}(y\|p) + \text{const}$.

Why $\sqrt{d_k}$: $q,k \sim \mathcal{N}(0, I_{d_k}) \Rightarrow q^\top k$ has var d_k; without scaling softmax saturates → vanishing grad on losing keys.

Skip-conn grad: $y = F(x)+x \Rightarrow \frac{\partial L}{\partial x} = \frac{\partial L}{\partial y}(I + \frac{\partial F}{\partial x})$ — identity always passes grad.

Dropout: inverted-dropout scales by $1/(1-p)$ at train so test forward is unchanged; else multiply weights by $(1-p)$ at test.

Adam vs SGD: SGD+mom often best on convnets (better generalize); Adam dominant on RNNs/Transformers (sparse/stiff grads); use AdamW for proper weight decay.

RF w/ strides: $\text{RF}_\ell = \text{RF}_{\ell-1} + (F_\ell - 1)\prod_{i<\ell} S_i$; effective stride $\prod S_i$; pad doesn't change RF.

Conv counting: VGG block 3×3, $C\to C$, $H\times W$: params $9C^2$, FLOPs $9C^2HW$, act CHW. 1×1 ($C\to C'$): params CC', FLOPs $CC'HW$.

IoU: $A=(x_1^a..y_2^a)$, B idem. $x_1 = \max(x_1^a, x_1^b), \dots, x_2 = \min(\dots)$; Inter $= \max(0, x_2-x_1)\cdot\max(0,y_2-y_1)$; Union $= |A|+|B|-\text{Inter}$.

NMS: `sort B by score; while B: b*=B.pop(0); keep+=[b*]; B={b: IoU(b,b*)<tau}`.

Anchors/RPN: $H_f W_f k$ proposals; cls head $2k$, reg head $4k$ per location.

Mixup: $\tilde{x} = \lambda x_i + (1-\lambda)x_j$, $\tilde{y} = \lambda y_i + (1-\lambda)y_j$, $\lambda\sim\text{Beta}(\alpha,\alpha)$. **CutMix**: paste rect of x_j into x_i, λ = area frac.

Optical flow: brightness const. $I_x u + I_y v + I_t = 0$; Lucas-Kanade locally; FlowNet/RAFT learn it.

Two-stream fusion: spatial CNN(RGB) + temporal CNN(stacked (u,v) over L frames, $2L$ ch); late avg.

GAN obj: $\min_G \max_D \mathbb{E}\log D(x) + \mathbb{E}\log(1-D(G(z)))$.

BN backward (full), with m=batch: $d\gamma = \sum_i dy_i \hat{x}_i$, $d\beta = \sum_i dy_i$, $d\hat{x}_i = \gamma\,dy_i$; $d\sigma^2 = -\frac{1}{2}\sum_i d\hat{x}_i(x_i-\mu)(\sigma^2+\varepsilon)^{-3/2}$; $d\mu = -\sum_i d\hat{x}_i(\sigma^2+\varepsilon)^{-1/2} - \frac{2}{m}d\sigma^2\sum_i(x_i-\mu)$; $dx_i = d\hat{x}_i(\sigma^2+\varepsilon)^{-1/2} + \frac{2}{m}d\sigma^2(x_i-\mu) + \frac{1}{m}d\mu$.

13 Diagnostics & Hyperparameter Tuning

Loss curves:

- Flat high → LR too low (or dead net).
- NaN / explodes → LR too high; clip grad; lower init.
- Train↓, val flat/up → overfit (more reg/data/aug).
- Train=val high → underfit (bigger model, longer train).
- Big train−val gap with high train acc → overfit.

Sanity: tiny subset (e.g. 20 ex.) should overfit to ∼0 loss. **Search**: random > grid; coarse-to-fine on log-scale ($\log\eta \sim \mathcal{U}[-5,-2]$, $\log\lambda\sim\mathcal{U}[-5,-1]$). **Track**: ratio $\|\Delta W\|/\|W\| \approx 10^{-3}$ healthy. **Param/activation**: monitor std after each layer; should not collapse to 0 or blow up.

13.1 Hyperparam defaults

LR: SGD 0.1 (cosine), Adam $3\cdot10^{-4}$; Momentum 0.9; WD $5\cdot10^{-4}$ (CNN), 0.05 (ViT, AdamW); Batch 256; Warmup 5 epochs; Cosine to 0.

14 Memory & Compute Quick Math

Activations dominate memory in CNNs (esp. early layers). Params dominate in FCs. **GPU mem** ≈ params (×4 fp32 or ×2 fp16) + activations (per batch) + optimizer states + workspace. **Activation size** per layer = $N{\cdot}C{\cdot}H{\cdot}W{\cdot}$ bytes. **Mixed precision**: forward/backward in fp16, master in fp32, loss scaling by S to avoid underflow.

15 Pitfalls

- kNN: hyperparams via val, not test; L_2 rotation-inv, L_1 axis-dependent.
- Softmax never 0; SVM hits 0 once margin satisfied.
- Sanity check loss $\approx \log C$ at init; ∇ on small W should match numeric grad.
- Shape rule: dW shape of W, dx shape of x.
- Max gate: grad → argmax only; ReLU passes if input>0 else 0.
- BN: train uses batch stats, test uses running; freeze BN with tiny batches.
- Same-pad needs odd F; otherwise asymmetric pad.
- Stack 3×3 > 7×7: fewer params ($27C^2$ vs $49C^2$), more nonlin.
- Vanish in RNN → LSTM/GRU; explode → clip.
- SA is $O(N^2)$; PE required (else permutation-equivariant).
- Two-stage (Faster R-CNN) accurate; one-stage (YOLO/SSD/RetinaNet) fast.
- Transposed conv ≠ inverse; checkerboard if $F\nmid S$.
- Adam without bias-correction biases low at start.
- Gradient clipping helps RNNs/transformers w/ exploding grads.
- BN+Dropout combined can hurt (variance shift); usually use one.